

# Padrão de interação com agentes no aplicativo gerado — versão 1.0

**Data:** 03/10/2026 · **Para:** o agente de Geração de Código do LangNet · **Status:** aprovado pelo autor em 03/10/2026

**O que mudou da 0.2:** as sete decisões confirmadas (seção 11); as três formas de auxílio por agente dentro das telas e onde cada bloco fica (seção 3.1); o menu lateral (seção 3.3); o lado convencional em rotas da API (seção 2.1); a versão do framework (seção 1).

## 1. O desenho

| Onde                        | O que é                                                                                                                                | Para quem                                       |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| LangNet (fábrica)           | a interface do visualtasksexec com as abas (rede de Petri, execução, verboses)                                                         | quem constrói: testa a orquestração dos agentes |
| Aplicativo gerado (máquina) | os <b>mesmos</b> agentes, a <b>mesma</b> rede de Petri e o <b>mesmo</b> framework (frameworkagentsadapter), com uma interface amigável | quem usa: médico, enfermeiro, administrador     |

**Framework:** o aplicativo usa o frameworkagentsadapterv5, a versão que orquestra por rede de Petri ("estrutura idêntica ao V4, mas usando Petri Net em vez de LangGraph"), depois de uma prova com a rede do BioByte. Se a prova falhar, fica a v4 (a que o TropicalSales e a fábrica já usam), com a rede disparando as tarefas. O console técnico da rede **não** vai para o aplicativo: fica só na Bancada do LangNet.

No aplicativo gerado, o usuário **nunca** vê lugar, transição, token, verbose nem JSON. Ele vê a pergunta, o que o sistema está fazendo em linguagem dele, a resposta e o porquê.

## 2. Três tipos de execução, declarados no caso de uso

Cada linha "Resposta do Sistema" do fluxo principal e dos fluxos de exceção traz **Executado por**:

| Valor         | O que é                                                    | Quem escreve                                                  | Roda IA?             |
|---------------|------------------------------------------------------------|---------------------------------------------------------------|----------------------|
| pronto        | login, cadastro, relatório, auditoria                      | molde determinístico do gerador                               | não                  |
| código gerado | regra própria do caso de uso que um programa comum resolve | o LLM, uma vez, na Geração de Código                          | não                  |
| agente        | interpretar, classificar, decidir, redigir                 | a tarefa no framework (CrewAI), definida em Agentes e Tarefas | sim, a cada execução |

Teste para separar código gerado de agente: *um programa comum, lendo o banco e seguindo uma regra escrita, resolveria?* Sim → código gerado. A Natureza do caso de uso passa a ser consequência: é agêntico se algum passo for agente.

### 2.1 Onde roda cada tipo

- pronto e código gerado → **rotas da API do backend gerado**: login com token (verificado em toda chamada), cadastro, consulta, relatório e as respostas escritas pelo LLM. Erro do banco volta como erro.
- agente → **servidor de agentes**, que é o framework carregando o `agents.yaml` e o `tasks.yaml`.

## 3. Os dois modos de interação com agente

### Modo A — agente embutido na tela de negócio

A tela do caso de uso continua sendo a tela do caso de uso (campos, tabelas, indicadores). O passo agente aparece **dentro** dela, como um bloco. É o modo padrão quando o caso de uso é uma tela de trabalho (ex.: UC-011, multirresistência; UC-013, redigir o alerta).

### Modo B — conversa (chat)

Um painel de conversa conduz uma **sequência** de passos de agente e de programa, tirada da rede de Petri: pergunta ao usuário só o que falta, executa, mostra o resultado de cada etapa e segue. É o modo para fluxos que atravessam vários casos de uso (ex.: "analisar o caso CAS-2023-001": tradução → classificação → multirresistência → recomendação → alerta).

### 3.1 As três formas de auxílio por agente dentro de uma tela (Modo A)

| Forma | Quando | Como aparece                                              | Exemplo |
|-------|--------|-----------------------------------------------------------|---------|
|       |        | botão + andamento + cartão de decisão com a justificativa |         |

| Forma                   | Quando                                                    | Como aparece                                                                                                           | Exemplo                                                |
|-------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Bloco de decisão</b> | o agente decide ou classifica algo que está na tela       |                                                                                                                        | "Identificar multirresistência" → Sim · 4 classes      |
| <b>Auxílio de campo</b> | o agente redige ou sugere o conteúdo de um campo          | botão "✦ Redigir com IA" / "✦ Sugerir" ao lado do campo; o texto entra no campo para o usuário revisar antes de salvar | justificativa da sobrescrita; texto do alerta          |
| <b>Ação da tela</b>     | o agente processa a tela inteira e preenche vários campos | botão no topo da tela; os campos preenchidos ficam destacados até o usuário confirmar                                  | "Traduzir laudo" preenche microrganismo e antibiograma |

**Onde o bloco fica:** no ponto da tela em que o passo está no fluxo do caso de uso (ex.: logo abaixo da tabela que ele analisa). A etapa de Interface declara a posição (depois de: <componente>) e a forma (forma: decisao | campo | tela). Uma tela convencional pode ter um auxílio de campo sem deixar de ser convencional.

### 3.2 Regra de escolha entre os modos

**Regra** (declarada na etapa de Interface, por tela): o caso de uso com um passo agente dentro de uma tela de trabalho → **Modo A**. A sequência da rede que começa num gatilho do usuário e passa por mais de um caso de uso → **Modo B**. O mesmo agente pode aparecer nos dois modos; a tarefa é uma só.

O Modo B é **um assistente único** do aplicativo: pergunta "o que você quer fazer?" e conhece os fluxos que a rede de Petri define. Começa com um fluxo (no BioByte, "analisar um caso") e ganha os outros.

### 3.3 O menu lateral

- **Sem seção separada de agentes de IA:** o menu continua agrupado por assunto (Atendimento, Relatórios, Cadastros...), e cada função com IA fica na tela do seu assunto.
- **O Assistente fica fixo no topo do menu.**
- **Telas com IA levam a marca ✦** ao lado do nome, gerada a partir dos passos agente do caso de uso.
- Dentro do Assistente, a página **"O que a IA faz neste sistema"** lista cada função com IA e leva à tela dela.

## 4. Onde entram o agents.yaml e o tasks.yaml

São a definição dos agentes e das tarefas que o framework executa, no formato do CrewAI usado no TropicalSales: o agents.yaml traz papel, objetivo, história e ferramentas de cada agente; o tasks.yaml, a descrição, a saída esperada e o agente de cada tarefa. A etapa **YAML** do LangNet gera os dois a partir do documento de Agentes e Tarefas.

#### O caminho deles

1. **Na fábrica:** a etapa YAML gera os dois; a Sequência e a Rede de Petri apontam para as tarefas **pelo nome** do tasks.yaml; a Bancada do LangNet testa a rede com eles carregados no framework.
2. **No aplicativo gerado:** os dois vão **do jeito que saíram da fábrica**. O servidor de agentes é o framework, que carrega os dois ao subir. A Geração de Código **não reescreve nem reinterpreta** os YAML: o que roda no hospital é exatamente o que foi testado na Bancada.
3. **Telas e chat só acionam pelo nome:** no Modo A, o botão dispara uma tarefa do tasks.yaml; no Modo B, a conversa dispara a rede, e cada lugar da rede chama a sua tarefa.

#### De onde sai cada campo da ficha de interação

| Campo da ficha              | Origem                                                  |
|-----------------------------|---------------------------------------------------------|
| tarefa                      | o nome da tarefa no tasks.yaml                          |
| agente que executa          | o campo agent da tarefa → o agente no agents.yaml       |
| entradas                    | o formato de entrada declarado na description da tarefa |
| saídas                      | as chaves declaradas no expected_output da tarefa       |
| ferramentas                 | as tools do agente no agents.yaml                       |
| origem das entradas, depois | a Sequência e a Rede de Petri                           |
| modo, componentes, rótulo   | a etapa de Interface & Protótipo                        |
| mensagens de recusa         | os fluxos de exceção do caso de uso                     |

**Portão:** toda tarefa citada numa tela ou no chat existe no tasks.yaml; as entradas e saídas da ficha batem com a description e o expected\_output dela; o agente da tarefa existe no agents.yaml. Se não bate, a geração não libera.

O que não vem dos YAML: os passos pronto e código gerado — o lado convencional, que não passa pelo framework.

### 5. A ficha de interação — o que o gerador recebe por passo agente

Montada pelo programa a partir das etapas anteriores (não é inventada pelo modelo):

```
acao: "Identificar multirresistência"           # rótulo do botão (Interface)
caso_de_uso: UC-011                             # Especificação
tarefa: evaluate_multidrug_resistance           # Agentes e Tarefas
modo: embutido                                  # embutido | conversa (Interface)
forma: decisao                                  # decisao | campo | tela (Interface)
depois_de: tabela_antibiograma                 # posição na tela (Interface)
entradas:                                       # cada entrada diz DE ONDE vem
  - campo: caso_id
    origem: contexto                           # contexto (caso/atendimento corrente) | tela (campo X) | perguntar | programa
    pergunta: "Qual caso clínico você quer avaliar?" # usada se o contexto estiver vazio
    escolher_de: casos_clinicos                 # lista para escolher (nunca digitar id)
  - campo: antibiograma_traduzido
    origem: programa                           # o programa busca no banco / no passo anterior; o usuário não vê
saídas:                                       # como cada campo de saída aparece
  - campo: multirresistente ; mostrar: decisao # selo grande: Sim / Não / Pendente
  - campo: classes_resistentes ; mostrar: lista
  - campo: contagem_classes_resistentes ; mostrar: indicador
  - campo: justificativa ; mostrar: texto      # sempre visível, nunca escondida
grava: { tabela: decisoes_sistema, mostrar_registro: true }
humano_no_meio: confirmar                     # nenhum | confirmar | revisar_e_editar
recusas:                                     # do fluxo de exceção do caso de uso
  - quando: "antibiograma sem classe"
    mensagem: "Classe ausente em um ou mais antimicrobianos."
  - quando: "caso não informado"
    mensagem: "Escolha um caso clínico para avaliar."
depois: [ "Redigir alerta (UC-013)" ]         # próxima ação sugerida, tirada da rede
```

### 6. Os componentes de interface (sempre os mesmos, em todo aplicativo)

| Componente              | Para quê                                 | Regras                                                                                                                                                                                                                     |
|-------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seletor de contexto     | escolher o caso/atendimento quando falta | lista com rótulo legível (paciente, data, situação); nunca campo de id                                                                                                                                                     |
| Botão de ação do agente | disparar o passo                         | rótulo igual ao do caso de uso; desabilitado só com o motivo escrito ao lado                                                                                                                                               |
| Andamento               | mostrar o que está acontecendo           | frases do usuário ("lendo o antibiograma...", "contando classes resistentes..."), tiradas dos passos da tarefa; sem verbose técnico e sem o raciocínio interno do modelo, que fica no registro e na Bancada                |
| Cartão de decisão       | mostrar o resultado                      | o veredito em destaque, os números, a justificativa sempre visível, a versão do critério, link "ver registro"                                                                                                              |
| Pergunta                | pedir uma entrada que falta              | a pergunta da ficha, com o tipo certo de campo (lista, data, número)                                                                                                                                                       |
| Confirmação             | humano no meio                           | Aceitar · Refazer com observação · Cancelar — a observação vai para o agente na nova tentativa. Obrigatória <b>antes de gravar uma decisão clínica ou de enviar algo para fora</b> ; ler e consultar não pedem confirmação |
| Recusa                  | o agente não pôde decidir                | a mensagem do caso de uso + o que falta para poder decidir; nunca "sucesso"                                                                                                                                                |
| Erro                    | falha técnica                            | mensagem clara + "tentar novamente"; o erro real vai para o registro, nunca some                                                                                                                                           |
| Próximo passo           | continuar o fluxo                        | botão para a ação seguinte da rede, levando o contexto                                                                                                                                                                     |

No **Modo B**, as mesmas peças viram mensagens da conversa: a pergunta é uma mensagem do sistema, o cartão de decisão é uma mensagem com o cartão, a confirmação são botões na mensagem.

### 7. Estados de toda interação (os dois modos)

aguardando entrada → executando → concluído | recusado | erro → (confirmado | refazer)

Toda tela e todo chat tratam **os cinco**. Resposta com status: erro ou recusa **nunca** aparece como sucesso.

8. De onde vem cada pedaço (as etapas do pipeline)

| Pedaço da ficha                                  | Etapas                    |
|--------------------------------------------------|---------------------------|
| Executado por de cada passo, mensagens de recusa | Especificação             |
| tela, rótulo da ação, modo, componentes          | Interface & Protótipo     |
| tarefa, entradas e saídas com tipo               | Agentes & Tarefas         |
| origem das entradas, depois                      | Sequência e Rede de Petri |
| grava                                            | Modelo de Dados           |

9. Regras para o agente de Geração de Código

- 1. Para cada passo agente, ler a ficha e gerar **só** com os componentes da seção 6.
- 2. Toda entrada tem origem; contexto vazio vira **Pergunta**, nunca valor vazio enviado ao agente.
- 3. Toda saída declarada aparece; a justificativa nunca fica escondida.
- 4. As mensagens de recusa são as do caso de uso, literais.
- 5. O agente roda no framework, pela mesma tarefa que a fábrica testou; nada de reimplementar a tarefa na tela.
- 6. Para cada passo código gerado, escrever a resposta como código comum, com as mensagens do caso de uso.
- 7. Para cada passo pronto, usar o molde.

**Portão (programa, não modelo):** toda ação de toda tela tem quem a execute; toda entrada tem origem; toda mensagem de exceção aparece no código; nenhuma resposta de erro é tratada como sucesso.

10. Exemplo — BioByte

- **Modo A, UC-011:** abre a tela → sem caso no contexto, aparece "Qual caso clínico você quer avaliar?" com a lista de casos → "Identificar multirresistência" → andamento "lendo o antibiograma... contando classes resistentes..." → cartão: **Multirresistente: Sim · 4 classes** (cefalosporina, betalactâmico, fluoroquinolona, aminoglicosídeo), justificativa visível, "ver registro" → botão "Redigir alerta".
- **Modo B, "analisar um caso":** "Qual caso?" → tradução (cartão) → classificação (cartão: *Pendente — faltam os critérios NHSN vigentes e a data de início do caso*) → a conversa diz o que falta e oferece continuar só com a multirresistência → alerta redigido → "Enviar à equipe?" (confirmação).

11. Decisões tomadas (03/10/2026)

| # | Decisão                                                                          | Decidido                                                                                                                                                             | Por quê                                                                                                                                                 |
|---|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | O chat é <b>um painel por fluxo</b> ou <b>um assistente único</b> do aplicativo? | <b>Assistente único</b> , que conhece os fluxos da rede e oferece "o que você quer fazer?"; começa com um fluxo (analisar um caso) e ganha os outros                 | o usuário não precisa saber qual fluxo abrir; uma só porta de entrada; o mesmo componente serve a todo aplicativo gerado                                |
| 2 | Quando a confirmação humana é obrigatória?                                       | <b>Antes de gravar uma decisão clínica ou de enviar algo para fora</b> (alerta, e-mail). Consultar e ler, sem confirmação                                            | a responsabilidade clínica fica com o profissional, como a especificação exige, sem obrigar a clicar em cada leitura                                    |
| 3 | O andamento mostra o <b>raciocínio</b> do agente ou só os <b>passos</b> ?        | <b>Só os passos</b> em linguagem de usuário, e a <b>justificativa</b> no resultado; o raciocínio completo fica no registro e na Bancada                              | o raciocínio cru é longo e técnico; a justificativa é o que o médico precisa para concordar ou contestar                                                |
| 4 | Qual versão do framework o aplicativo usa?                                       | <b>frameworkagentsadapterv5</b> (orquestração por rede de Petri), depois de provada com a rede do BioByte; se falhar, v4                                             | a rede que a fábrica gera é a orquestração que o framework executa, sem reimplementação                                                                 |
| 5 | Onde roda o lado convencional (pronto e código gerado)?                          | <b>Rotas da API no backend gerado</b> (login com token, cadastro, consulta, relatório e as respostas geradas pelo LLM); o servidor de agentes fica só com o que é IA | login e segurança no lugar certo; não depende do servidor de agentes estar no ar; é programa comum, testável sem modelo e sem custo                     |
| 6 | A marca "Executado por" vai em <b>cada passo</b> do caso de uso?                 | <b>Sim</b> , nos fluxos principal e de exceção; a Natureza do caso de uso passa a ser consequência                                                                   | um caso de uso mistura os três tipos (o UC-011 tem consulta, regra fechada e decisão do agente); marcar o caso inteiro fez sumir a resposta de 20 telas |
| 7 | O console técnico da rede ("Admin / Petri") sai do aplicativo gerado?            | <b>Sim:</b> o console técnico fica só na Bancada do LangNet                                                                                                          | o usuário do hospital não opera rede de Petri; a cópia reduzida é código a manter sem uso                                                               |

| # | Decisão                            | Decidido                                                                                                 | Por quê                                                                              |
|---|------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 8 | Seção de agentes de IA no menu?    | <b>Não:</b> menu por assunto, Assistente fixo no topo, marca ✦ nas telas com IA, página "O que a IA faz" | o usuário procura a tarefa, não a tecnologia; a tarefa não se divide em dois lugares |
| 9 | Formas de auxílio dentro das telas | <b>Bloco de decisão, auxílio de campo e ação da tela</b> , na posição do passo no fluxo                  | cobre decisão, redação e preenchimento sem criar telas novas                         |